CSE 144 Final Project Report

Transfer Learning Challenge

Group Members:

Hansa Atreya (CruzID: hatreya)

Zhiyuan Song (CruzID: zsong41)

Paola Alvarez (CruzID: palvare9)

Spring 2026

1 Introduction

This project addresses a 100-class image classification challenge on a small custom dataset hosted on Kaggle (UCSC CSE 144 Spring 2026 Final Project). The dataset is sampled from multiple source datasets to form 100 classes, with exactly 10 training images and 10 evaluation images per class (1,000 training images and 1,000 test images total). Images are colored but vary in resolution, requiring standardization before training.

Since there are only 10 labelled images per class, it would be infeasible to train a deep network from scratch with the given data. It would not be possible for the model to learn meaningful visual information. Therefore, using transfer learning with pre-trained ImageNet weights allows for the model to use general feature knowledge like edges, shapes, and texture learned on large datasets, and then apply that knowledge to learning the specific classes of our dataset, dramatically reducing overfitting.

We fine-tuned a ResNet-18 backbone pretrained on ImageNet, replacing the final classification head with a linear layer mapping to 100 classes. Training used the Adam optimizer at lr=1e-4 for 50 epochs with data augmentation, such as random greyscale, flipping, cropping, rotation, and color jitter. Our best Kaggle public leaderboard score was 0.75454, exceeding the required 60% baseline.

2 Dataset

The dataset contains 100 classes with 10 training images per class (1,000 training images total) and a held-out test split used for Kaggle evaluation. All images are RGB but vary in original resolution.

The training data is organized as train/<class_id>/*.jpg, where <class_id> is an integer string from "0" to "99". Because torchvision.datasets.ImageFolder assigns labels by lexicographic sort of folder names ("0", "1", "10", "100", "11", ...), the default indices do not match the Kaggle label space. We therefore remap the labels back to the folder integer so that the model output index directly corresponds to the Kaggle label:

dataset.class_to_idx = {cls: int(cls) for cls in dataset.classes}
Dataset.targets = [int(dataset.classes[t]) for t in dataset.targets]
dataset.samples = [(s, int(dataset.classes[t])) for s, t in dataset.samples]
All images are resized to 224×224 and normalized with the ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) to match the pretrained backbone's input distribution. At training time we additionally apply the following data augmentation pipeline to combat overfitting on the small training set:

- RandomHorizontalFlip()
- RandomRotation(15)
- RandomCrop(224, padding=16)
- ColorJitter(brightness=0.2, contrast=0.2)
- RandomGrayscale(p=0.1)

At inference time we use only Resize(224) + ToTensor + Normalize (no stochastic operations) so predictions are deterministic.

3 Implementation

3.1 Model

We use ResNet-18 pretrained on ImageNet as the backbone, loaded via torchvision.models.resnet18(weights=ResNet18_Weights.DEFAULT). The only architectural change is replacing the final fully-connected layer with a new linear layer that maps the 512-dim feature vector to our 100 classes:

model.fc = nn.Linear(model.fc.in_features, 100)
We chose ResNet-18 over deeper / larger backbones (we briefly experimented with EfficientNet-B0) because with only 1,000 training images, a smaller model with fewer parameters is less prone to overfitting and still benefits strongly from the ImageNet feature

initialization. No layers are frozen — the entire network is fine-tuned end-to-end at a low learning rate so the pretrained features are preserved rather than overwritten.

3.2 Training

Setting	Value
Loss Function	Cross-Entropy Loss
Optimizer	Adam
Learning Rate	1e-4
Weight Decay	0
Learning Rate Scheduler	None
Batch Size	32
Epochs	50
Random Seed	42

Training and inference are performed in a single script (train.py) on a Kaggle Notebook with an NVIDIA T4 GPU. The software stack is PyTorch ≥ 2.0 with torchvision ≥ 0.15 (full list in requirements.txt). At the end of training, the model is switched to eval() mode and run on every image in the test folder to produce submission.csv.

4 Experiments

Baseline. Our initial baseline used ResNet-18 with only Resize + Normalize (no augmentation), Adam at lr=1e-3, and 10 epochs. This reached 0.518 on the Kaggle public leaderboard and showed clear overfitting (training accuracy approached 1.0 within a few epochs).

Validation strategy. Because each class contains only 10 training images, splitting off a validation set would leave at most 1- 2 images per class, too few to be statistically meaningful. We therefore trained on the full 1,000-image training set (val_split=0.0) and used the Kaggle public leaderboard as our validation signal. With the daily submission limit of 3, we monitored training-set accuracy locally for convergence and used each LB submission to decide which direction to push next.

Ablations. Each change below was evaluated against a single Kaggle submission and kept only if it improved the public LB score.

#	Change from previous	Public LB
1	Baseline (ResNet-18, no aug, lr=1e-3, 10 ep)	0.518
2	Swap backbone to EfficientNet-B0	0.53636
3	Revert to ResNet-18 + full augmentation + lr → 1e-4, 20 ep	0.700
4	Add `RandomGrayscale(p=0.1)`	0.71818
5	Add StepLR scheduler	0.664 (worse)
6	Add CutMix augmentation	did not converge
7	Remove CutMix, extend epochs 20 → 40	0.74545
8	Extend epochs to 40 → 50	0.75454

The two largest gains came from (a) reducing the learning rate from 1e-3 to 1e-4, which preserves the ImageNet features instead of overwriting them, and (b) the augmentation pipeline, which slowed memorization of the 1,000 training images. The LR scheduler and CutMix experiments were dropped: the scheduler decayed an already-small learning rate into the underfitting regime, and CutMix-style label mixing was too aggressive given only 10 images per class.

5 Results

Our best Kaggle public leaderboard score is **0.75454**, achieved with ResNet-18 pretrained on ImageNet, the full augmentation pipeline, the Adam optimizer at lr=1e-4, and 50 training epochs.

To report averaged metrics over multiple runs, we repeated the final configuration with three random seeds (42, 43, 44), obtaining Kaggle public scores of 0.75454, 0.72727, and 0.72727 respectively (mean 0.7364, std 0.0157). The committed code uses seed 42, which produced our best score. All runs exceed the 60% baseline by a wide margin.

The ablation progression is summarized in the table in Section 4. The two largest gains came from reducing the learning rate from 1e-3 to 1e-4 (0.518 to 0.700) and the augmentation pipeline, which slowed memorization of the 1,000 training images. Extending training from 20 to 40 to 50 epochs gave further improvement (0.71818 to 0.74545 to 0.75454). Interventions that did not help include StepLR scheduling (0.664), CutMix augmentation (did not converge), and label smoothing combined with test-time augmentation (0.68181).

To visualize the training dynamics, we ran a separate diagnostic session using an 80/20 train/validation split with seed 42. Training accuracy rose rapidly, reaching roughly 99.9% by epoch 12, while validation accuracy plateaued around 56–57%. This gap confirms that

overfitting is the primary bottleneck: with only 10 images per class, the model memorizes the training set rather than learning broadly generalizable features. The validation accuracy from this diagnostic split is not directly comparable to the Kaggle score, since the final submission model is trained on all 1,000 training images with no holdout.

6 Discussion

What worked best. The single most important decision was using a low learning rate (1e-4) for fine-tuning. At lr=1e-3, the early gradient updates were large enough to overwrite the pretrained ImageNet features, negating the benefit of transfer learning. Dropping to 1e-4 preserved those features while still allowing the network to adapt to the 100-class task. The augmentation pipeline was the second most important factor: with only 10 images per class, the model memorizes the training set within a few epochs unless augmentation introduces enough variation to slow that down. Training for longer (up to 50 epochs) continued to yield small improvements, indicating the model was still refining useful structure well past the point where training accuracy saturated.

What did not work. Several regularization techniques that are effective on larger datasets did not help here. CutMix, which blends patches from two images and mixes their labels, failed to converge, likely because mixing labels across only 10 examples per class introduces too much noise. StepLR scheduling decayed the already-small learning rate into a regime where the model stopped making progress. Label smoothing (0.1) combined with test-time augmentation scored 0.68181, below the baseline configuration, suggesting that further softening an already-weak training signal hurt more than it helped.

Limitations. The primary limitation is dataset size. With 1,000 training images, there is not enough data to hold out a statistically meaningful validation set (only 1 - 2 images per class), so we relied on the Kaggle public leaderboard as our validation signal, limited to 3 submissions per day. This makes systematic hyperparameter search slow and imprecise. A larger training set or few-shot learning techniques would improve both the reliability of validation and the final accuracy. Promising next steps include larger backbones (ResNet-34/50) and explicit weight-decay regularization.

7 Reproducibility

All training runs use a fixed random seed of 42 with full deterministic settings, configured in utils.py before any model or data operations. The seeding routine sets:

- os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8" (must be set before enabling deterministic algorithms)
- os.environ["PYTHONHASHSEED"] = "42"
- random.seed(42), np.random.seed(42), torch.manual_seed(42), torch.cuda.manual_seed_all(42)
- torch.backends.cudnn.benchmark = False
- torch.backends.cudnn.deterministic = True
- torch.use_deterministic_algorithms(True, warn_only=True)

The pipeline was independently reproduced by a second team member, who cloned the repository into a fresh Kaggle notebook and obtained the same score from the committed code without modification.

Environment: Kaggle Notebook with NVIDIA T4 GPU, PyTorch ≥ 2.0, torchvision ≥ 0.15 (full list in requirements.txt).

To reproduce training and generate predictions:

1. Open a Kaggle notebook with the competition dataset attached (mounted at /kaggle/input/competitions/ucsc-cse-144-spring-2026-final-project).
2. Clone the repository: git clone https://github.com/Lamuymuy/cse144-final-project
3. Run python train.py. The script trains the model for 50 epochs and writes submission.csv to the working directory.

To run inference from saved weights without retraining: Run python predict.py, pointing it at the downloaded weights file and the test data path. It loads the model, runs inference on all test images, and writes submission.csv.

Trained model weights are hosted on Google Drive with public access so inference can be run without retraining.

The final configuration was run independently twice (original run and reproduction from committed code), both producing identical predictions and the same leaderboard score of 0.75454, confirming reproducibility.

8 Team Contributions

1. Hansa Atreya (hatreya): Designed and implemented dataset.py(later absorbed into train.py). Authored utils.py with deterministic seeding that was used by the training runs. Drove the majority of training experiments and ablations via train.py: EfficientNet-B0 ablation (0.51818 → 0.53636), reversion to Resnet-18, added RandomGrayscale augmentation (0.518 → 0.71818); tested and removed StepLR scheduling (hurt score to 0.664); implemented and removed CutMix (did not converge); extended epochs from 20 → 30 → 40 → 50 (0.71818 → 0.75454); tested a freeze-then-unfreeze schedule (removed, hurt score); and ran the final ablation of label

smoothing + TTA (did not improve). Also optimized the training loop to use a single forward pass per batch, reducing compute time. Organized Github repo structure and README.
2. Zhiyuan Song (zsong41) — Refactored train.py to use the shared utilities in utils.py (deterministic seeding via set_seed, device selection via get_device) and the data loading in dataset.py. Designed and implemented the training-time augmentation pipeline (HorizontalFlip + Rotation + RandomCrop + ColorJitter) and the lr = 1e-4 fine-tuning configuration that first crossed the 60% baseline (0.518 → 0.700).
3. Paola Alvarez (palvare9): Verified reproducibility of the best score from committed code, wrote the inference script (predict.py) and project README, uploaded trained model weights to Google Drive, ran label smoothing + TTA ablation, and produced the training/validation accuracy curves.

9 References

1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR 2016. https://arxiv.org/abs/1512.03385
2. TorchVision Models and Pretrained Weights. PyTorch Documentation. https://pytorch.org/vision/stable/models.html
3. TorchVision ResNet18 pretrained model documentation. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html
4. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019. https://arxiv.org/abs/1905.11946
5. TorchVision EfficientNet_B0 pretrained model documentation. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.efficientnet_b0.html
6. Yun, S., et al. (2019). CutMix: Training Strategy that Makes Use of Sample Pairs. ICCV 2019. https://arxiv.org/abs/1905.04899
7. PyTorch Documentation — torch.utils.data, torchvision.transforms, torch.optim.Adam. https://pytorch.org/docs/stable/index.html
8. Kalantar, Reza. (2023). How to Freeze Model Weights in PyTorch for Transfer Learning: Step-by-Step Tutorial. https://python.plainenglish.io/how-to-freeze-model-weights-in-pytorch-for-transfer-learning-step-by-step-tutorial-a533a58051ef